4.3 Building the Hybrid Machine Learning Model

4.3.1 Hybrid Model Overview

To further improve collision risk prediction performance, a hybrid modelling approach was developed by combining regression and classification models. The hybrid architecture leverages the strengths of both modelling approaches. The regression stage predicts continuous log-scaled collision probability values, while the classification stage refines predictions by identifying high-risk conjunction events. This two-stage pipeline improves detection reliability and reduces the likelihood of missing dangerous collisions.

Based on the classification results, the LightGBM classifier achieved the highest overall performance across accuracy, precision, recall, F1, and F2 scores, while maintaining moderate runtime. Due to its strong performance and reliability in detecting high-risk conjunction events, LightGBM was selected as the primary classification component for the hybrid modelling pipeline.

Multiple hybrid combinations were then developed by pairing top-performing regression models with the selected classification models. These combinations were evaluated to identify the most effective hybrid configuration for collision risk prediction.

4.3.2 Hybrid Model Architecture

The hybrid architecture consists of two stages:

- **Regression Stage:** Predicts continuous log-scaled collision probability values for each conjunction event
- **Classification Stage:** Refines predictions by classifying events into risk categories while emphasising recall for high-risk detection

This approach improves prediction reliability by combining continuous probability estimation with discrete risk classification.

4.3.3 Hybrid Models Tested

Multiple hybrid combinations were implemented and evaluated. These combinations were created by pairing top-performing regression models with classification models.

ADD TABLE

4.3.4 Hybrid Model Evaluation

The hybrid approach combines:

Regression stage: predicts continuous log-scaled collision probability values for each conjunction event.

Classification stage: refines high-risk detection by classifying events into high-risk or low-risk categories while emphasizing recall.

$$F_2 = (1 + 2^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{4 \cdot \text{Precision} + \text{Recall}}$$

ADD TABLE

Because the hybrid approach combines continuous regression outputs with classification-based predictions, some evaluation metrics that require purely continuous values become unsuitable. In particular, the MSEHR and L-metric rely on continuous regression outputs and therefore cannot be directly applied to hybrid predictions.

As a result, the F2 score was selected as the primary evaluation metric for the hybrid models. The F2 score places greater emphasis on recall, making it particularly appropriate for collision risk prediction. In this context, failing to detect a high-risk conjunction is significantly more critical than generating a false alert, making recall-focused evaluation essential for ensuring operational safety.

The hybrid approach is also benchmarked against the Latest Risk Prediction (LRP) baseline, which is commonly used in operational satellite conjunction assessment systems. To ensure comparability with regression-only models, multiple evaluation metrics are reported, including the F2 score, MSEHR, and the L-metric. This allows consistent performance comparison between hybrid and traditional regression-based approaches, providing a more complete assessment of model effectiveness in collision risk prediction.

4.3.4 Code used:

Code Section 1: Library Imports and Setup

The hybrid modelling pipeline begins by importing essential libraries required for data processing, machine learning, and evaluation. NumPy and Pandas are used for numerical computation and dataset manipulation. Scikit-learn provides evaluation metrics and preprocessing tools, while ensemble learning models such as Random Forest, XGBoost, LightGBM, and CatBoost are used for both regression and classification tasks. Additional models such as MLPClassifier are used for deep learning-based classification. Parallel processing tools are also imported to improve execution efficiency during model training.

```
import numpy as np
import pandas as pd
import time

from concurrent.futures import ThreadPoolExecutor, as_completed

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, fbeta_score
)

# Models
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from xgboost import XGBRegressor, XGBClassifier
from lightgbm import LGBMRegressor, LGBMClassifier
from catboost import CatBoostRegressor
from sklearn.neural_network import MLPClassifier
```

Figure 4.3.4.1.1: Library Imports

Code Section 2: Data Loading and Overview

The training and testing datasets are loaded from the preprocessed data directory. These datasets contain engineered satellite conjunction features and the target risk variable. After loading, the dataset shapes are printed to verify successful import and ensure consistency between training and testing data.

```python
# ====================================================
# LOAD DATA
# ====================================================

train_data = pd.read_csv("../data/raw/train_data.csv")
test_data = pd.read_csv("../data/raw/test_data.csv")

print("Train shape:", train_data.shape)
print("Test shape:", test_data.shape)
```

Figure 4.3.4.2.1: Data Loading Code

Code Section 3: Target Variable Preparation (Regression + Classification)

The hybrid model uses a dual-target approach. The original continuous risk values are used for regression modelling, while a custom function converts these values into discrete risk classes for classification.

The classification function divides risk into three categories:

- Class 0: Low Risk
- Class 1: Medium Risk
- Class 2: High Risk

This ensures that the model can both predict continuous probabilities and classify risk levels simultaneously.

```python
# TARGETS
# ====================================================

def create_risk_class(y):
    return np.select(
        [y > -4, (y <= -4) & (y > -6), y <= -6],
        [0, 1, 2]
    ).astype(np.int8)

y_train_class = create_risk_class(train_data["risk"])
y_test_class = create_risk_class(test_data["risk"])

y_train_reg = train_data["risk"].values
y_test_reg = test_data["risk"].values
```

Figure 4.3.4.3.1: Target Creation Function

Code Section 4: Feature Engineering and Data Cleaning

Feature preparation begins by separating input features from the target variable. Categorical variables are encoded using one-hot encoding to convert them into numerical format. The training and testing datasets are then aligned to ensure both contain identical feature columns.

To improve data quality, infinite values are replaced with NaN, and missing values are filled using median imputation. Feature values are also clipped within a defined range to prevent extreme values from affecting model stability. Finally, data is converted into NumPy arrays for faster computation and reduced memory usage.

This step is critical to ensure numerical stability across both regression and classification models.

```python
# FEATURES + CLEANING (CRITICAL FIX)
# ==================================================

X_train = pd.get_dummies(train_data.drop(columns=["risk"]))
X_test = pd.get_dummies(test_data.drop(columns=["risk"]))

X_train, X_test = X_train.align(X_test, join="left", axis=1, fill_value=0)

X_train = X_train.replace([np.inf, -np.inf], np.nan)
X_test = X_test.replace([np.inf, -np.inf], np.nan)

median_vals = X_train.median()

X_train = X_train.fillna(median_vals)
X_test = X_test.fillna(median_vals)

# 🔥 Prevent overflow
X_train = X_train.clip(-1e6, 1e6)
X_test = X_test.clip(-1e6, 1e6)

# Convert to NumPy (FASTER)
X_train = X_train.values.astype(np.float32)
X_test = X_test.values.astype(np.float32)

# Final safety
X_train = np.nan_to_num(X_train, posinf=1e6, neginf=-1e6)
X_test = np.nan_to_num(X_test, posinf=1e6, neginf=-1e6)
```

Figure 4.3.4.4.1: Feature Engineering and Cleaning Code

Code Section 5: Feature Scaling

Standardisation is applied to the dataset using StandardScaler. Scaling ensures that all features contribute equally during model training, particularly for neural network-based classifiers. Scaling is applied only where required, ensuring that tree-based models are not affected unnecessarily.

```python
# ==================================================
# SCALING (ONLY FOR DNN)
# ==================================================

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 4.3.4.5.1: Feature Scaling Code

Code Section 6: Hybrid Evaluation Function

An evaluation function is defined to assess hybrid model performance. It calculates key metrics including accuracy, precision, recall, F1 score, and F2 score. The F2 score is particularly important as it prioritises recall, which is critical in satellite collision prediction where missing high-risk events is more severe than false alarms.

All evaluation results are stored in a structured list for later comparison across models.

```python
# RESULTS STORAGE
# ====================================================
hybrid_results = []

def evaluate(name, y_true, y_pred, runtime):
    hybrid_results.append({
        "Hybrid Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, average="macro", zero_division=0),
        "Recall": recall_score(y_true, y_pred, average="macro", zero_division=0),
        "F1": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "F2": fbeta_score(y_true, y_pred, beta=2, average="macro", zero_division=0),
        "Runtime": runtime
    })
    print(f"{name} completed in {runtime:.2f} sec")

# ====================================================
# HYBRID FUNCTION
# ====================================================
def run_hybrid(name, reg_model, clf_model):

    start = time.time()

    reg_model.fit(X_train, y_train_reg)
    reg_train = reg_model.predict(X_train)
    reg_test = reg_model.predict(X_test)

    X_train_h = np.column_stack((X_train, reg_train))
    X_test_h = np.column_stack((X_test, reg_test))

    clf_model.fit(X_train_h, y_train_class)
    pred = clf_model.predict(X_test_h)

    evaluate(name, y_test_class, pred, time.time() - start)
```

Figure 4.3.4.6.1: Evaluation Function Code

Code Section 7: Hybrid Model Logic (Core Architecture)

The core hybrid function implements a two-stage architecture:

- Regression Stage: A regression model is trained on the dataset to predict continuous collision risk values.
- Feature Augmentation: The regression predictions are appended to the original feature set, enhancing the feature space.
- Classification Stage: A classification model is trained using the augmented dataset to predict risk classes.

This structure allows the classification model to benefit from regression-based risk estimates, improving overall prediction accuracy and sensitivity to high-risk events.

```python
# ====================================================
# HYBRID FUNCTION
# ====================================================
def run_hybrid(name, reg_model, clf_model):

    start = time.time()

    reg_model.fit(X_train, y_train_reg)
    reg_train = reg_model.predict(X_train)
    reg_test = reg_model.predict(X_test)

    X_train_h = np.column_stack((X_train, reg_train))
    X_test_h = np.column_stack((X_test, reg_test))

    clf_model.fit(X_train_h, y_train_class)
    pred = clf_model.predict(X_test_h)

    evaluate(name, y_test_class, pred, time.time() - start)
```

Figure 4.3.4.7.1: Hybrid Model Function Code

Code Section 8: Classical Hybrid Models

Multiple classical hybrid combinations are implemented to evaluate different regression–classification pairings. These include combinations of Random Forest, XGBoost, LightGBM, and CatBoost models.

The tested hybrid configurations are:

- RF + LightGBM
- RF + XGBoost
- LightGBM + LightGBM
- XGBoost + LightGBM
- CatBoost + LightGBM
- RF + RF

Parallel processing is used to execute multiple hybrid models simultaneously, improving computational efficiency.

```python
# ====================================================
# MODEL DEFINITIONS FOR HYBRIDS
# ====================================================

hybrid_models = [

    ("RF + LightGBM",
     RandomForestRegressor(n_estimators=120, n_jobs=1),
     LGBMClassifier(n_estimators=250, n_jobs=1)
    ),
    ("RF + XGBoost",
     RandomForestRegressor(n_estimators=120, n_jobs=1),
     XGBClassifier(n_estimators=250, n_jobs=1, tree_method="hist")
    ),
    ("LightGBM + LightGBM",
     LGBMRegressor(n_estimators=250, n_jobs=1),
     LGBMClassifier(n_estimators=250, n_jobs=1)
    ),
    ("XGBoost + LightGBM",
     XGBRegressor(n_estimators=250, n_jobs=1, tree_method="hist"),
     LGBMClassifier(n_estimators=250, n_jobs=1)
    ),
    ("CatBoost + LightGBM",
     CatBoostRegressor(iterations=250, verbose=0, thread_count=1),
     LGBMClassifier(n_estimators=250, n_jobs=1)
    ),
    ("RF + RF",
     RandomForestRegressor(n_estimators=120, n_jobs=1),
     RandomForestClassifier(n_estimators=120, n_jobs=1)
    )
]
```

Figure 4.3.4.8.1: Classical Hybrid Models Definition Code

Code Section 9: Deep Hybrid Models

In addition to classical models, deep learning-based hybrid models are implemented. These models use regression outputs as additional input features for a neural network classifier.

The deep hybrid models include:

- RF + DNN (MLP Classifier)
- LightGBM + DNN (MLP Classifier)

These models combine the predictive power of ensemble regression with the non-linear learning capability of neural networks.

```python
# ====================================================
# DEEP HYBRIDS (DNN)
# ====================================================

def run_dl(name, reg_model):

    start = time.time()

    reg_model.fit(X_train_scaled, y_train_reg)

    reg_train = reg_model.predict(X_train_scaled)
    reg_test = reg_model.predict(X_test_scaled)

    X_train_h = np.column_stack((X_train_scaled, reg_train))
    X_test_h = np.column_stack((X_test_scaled, reg_test))

    clf = MLPClassifier(
        hidden_layer_sizes=(64, 32),
        max_iter=100,
        early_stopping=True
    )

    clf.fit(X_train_h, y_train_class)
    pred = clf.predict(X_test_h)

    evaluate(name, y_test_class, pred, time.time() - start)

run_dl("RF + DNN", RandomForestRegressor(n_estimators=120, n_jobs=1))
run_dl("LightGBM + DNN", LGBMRegressor(n_estimators=250, n_jobs=1))
```

Figure 4.3.4.9.1: Deep Hybrid Model Code

Code Section 10: Final Results Generation

After training all hybrid models, the results are compiled into a structured dataframe. The models are sorted based on F2 score to prioritise recall performance. The final results table is then exported as a CSV file for further analysis and comparison.

```
# ====================================================
# FINAL RESULTS
# ====================================================

hybrid_df = pd.DataFrame(hybrid_results).round(6)
hybrid_df = hybrid_df.sort_values(by="F2", ascending=False)

print("\n============================")
print("HYBRID MODEL PERFORMANCE")
print("============================")
print(hybrid_df)

hybrid_df.to_csv("table_hybrid_models.csv", index=False)
```

Figure 4.3.4.10.1: Results Table Code